

Natural Computing

LMU Munich
summer term 2025

Thomas Gabor



Stellenangebote am Lehrstuhl für Mobile und Verteilte Systeme

Fullstack Entwickler (m/w/d)

- Fullstack- & App-Entwicklung mit Next.js, React, Expo
- Umsetzung kompletter Entwicklungszyklen
- Deployment auf Kubernetes
- Performanceoptimierung & Qualitätssicherung
- Kontinuierliches Upskilling in modernen Webtechnologien

Wochenstunden	9 - 19 h
Befristung	wiederkehrend 3 Monate (später ggf. längere Laufzeit)



Data Scientist (m/w/d)

- Datenanalyse im Gesundheitskontext
- Entwicklung von Analysepipelines in Python
- Datenvisualisierung mit Matplotlib & Co.
- Toolentwicklung & Testing
- Unterstützung bei Auswertung & Dokumentation

Wochenstunden	9 - 19 h
Befristung	wiederkehrend 3 Monate (später ggf. längere Laufzeit)



Announcement:

Python Crash Course

on Thursday, May 8

target audience:

students who are familiar with another programming language
and want to catch up on Python specifics

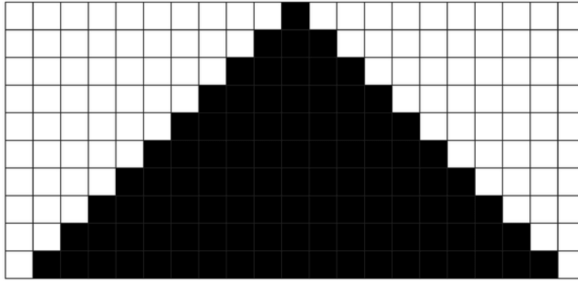
self-study material on basics available on Moodle! (look for “Exercise 0”)

Local information
is a constraint on
what we can do
efficiently

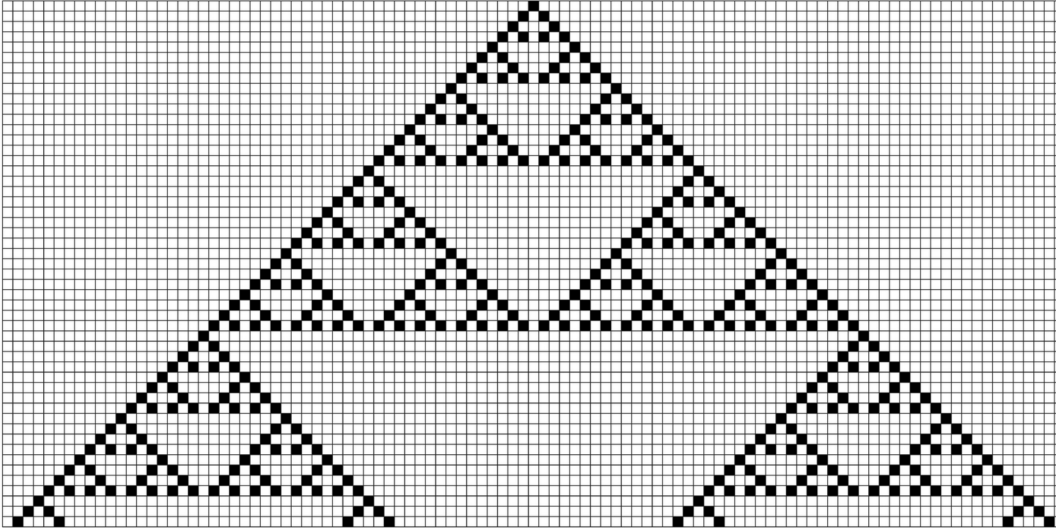
Is local information sufficient
for meaningful computation?

When I can use
only local information,
I might be more ...
- physical - robust

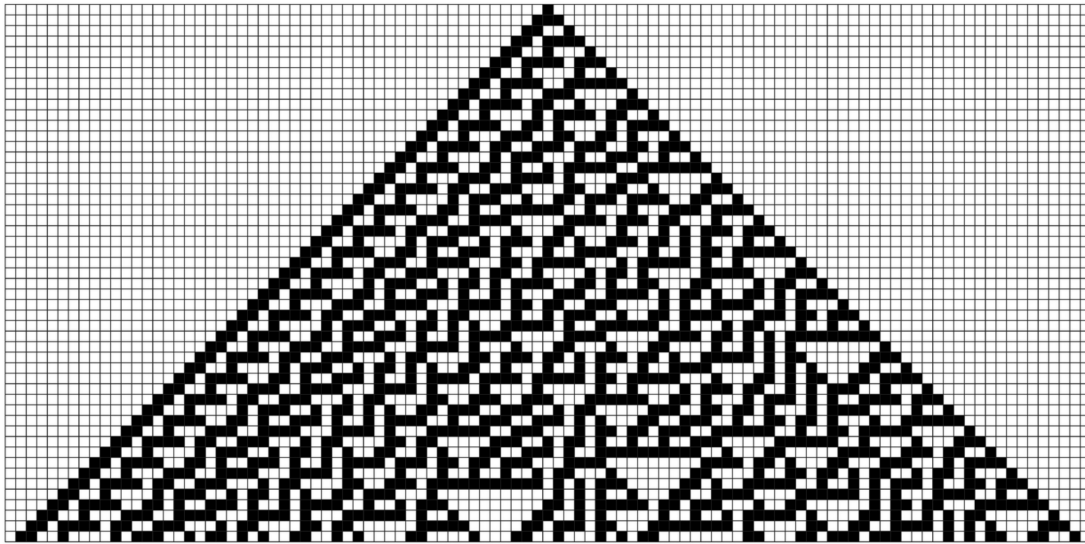
source:
wolframscience.com/nks



A representation of the rule for the cellular automaton shown above. The top row in each box gives one of the possible combinations of colors for a cell and its immediate neighbors. The bottom row then specifies what color the center cell should be on the next step in each of these cases. In the numbering scheme described in Chapter 3, this is cellular automaton rule 254.



A cellular automaton that produces an intricate nested pattern. The rule in this case is that a cell should be black whenever one or the other, but not both, of its neighbors were black on the step before. Even though the rule is very simple, the picture shows that the overall pattern obtained over the course of 50 steps starting from a single black cell is not so simple. The particular rule used here can be described by the formula $a'_i = \text{Mod}[a_{i-1} + a_{i+1}, 2]$. In the numbering scheme of Chapter 3, it is cellular automaton rule 90.



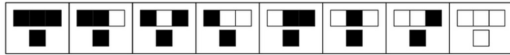
A cellular automaton with a simple rule that generates a pattern which seems in many respects random. The rule used is of the same type as in the previous examples, and the cellular automaton is again started from a single black cell. But now the pattern that is obtained is highly complex, and shows almost no overall regularity. This picture is our first example of the fundamental phenomenon that even with simple underlying rules and simple initial conditions, it is possible to produce behavior of great complexity. In the numbering scheme of Chapter 3, the cellular automaton shown here is rule 30.

On Names for Rules



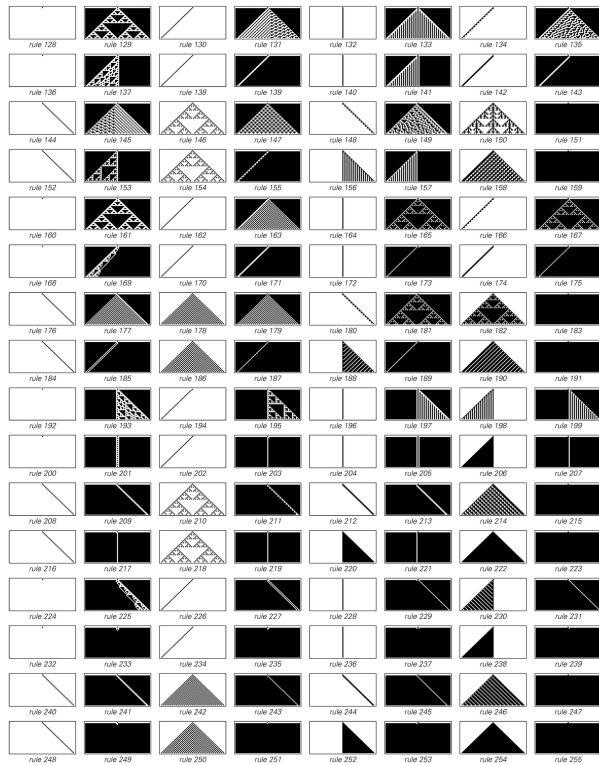
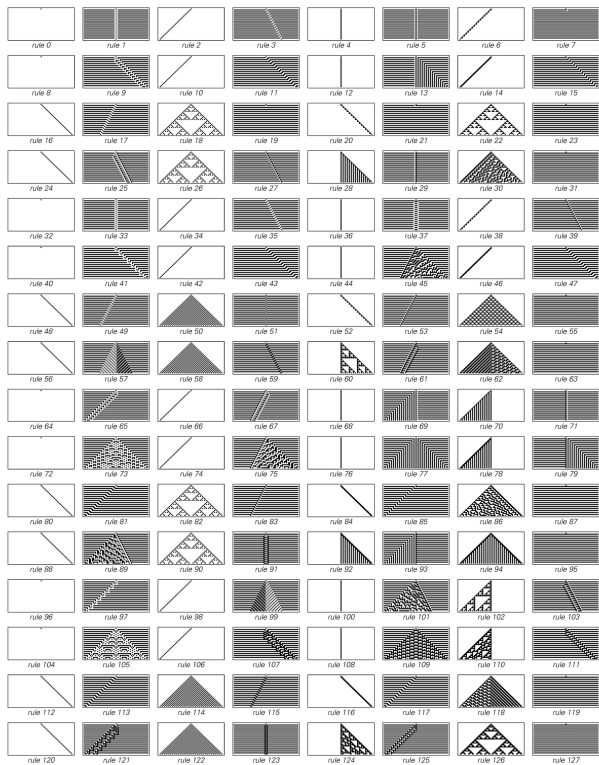
A cellular automaton that produces an intricate nested pattern. The rule in this case is that a cell should be black whenever one or the other, but not both, of its neighbors were black on the step before. Even though the rule is very simple, the picture

shows that the overall pattern obtained over the course of 50 steps starting from a single black cell is not so simple. The particular rule used here can be described by the formula $a_i' = \text{Mod}[a_{i-1} + a_{i+1}, 2]$. In the numbering scheme of Chapter 3, it is cellular automaton rule 90.



A representation of the rule for the cellular automaton shown above. The top row in each box

gives one of the possible combinations of colors for a cell and its immediate neighbors. The bottom row then specifies what color the center cell should be on the next step in each of these cases. In the numbering scheme described in Chapter 3, this is cellular automaton rule 254.



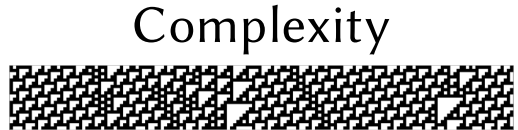
Classes of Cellular Automata



Uniformity



Repetition



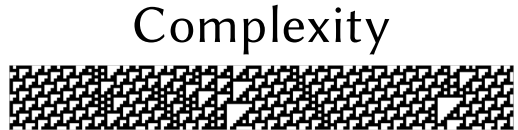
Classes of Cellular Automata



Uniformity



Repetition





source: commons.wikimedia.org/wiki/File:Textile_cone.JPG

Outlook: Neural Cellular Automata

Outlook: Neural Cellular Automata



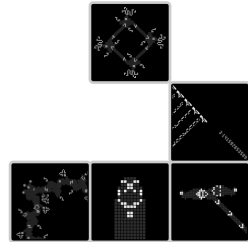
Let's try

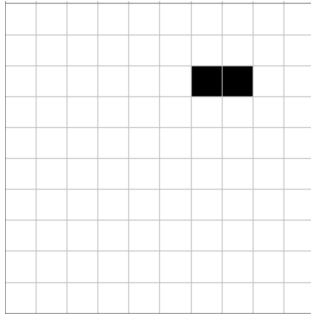
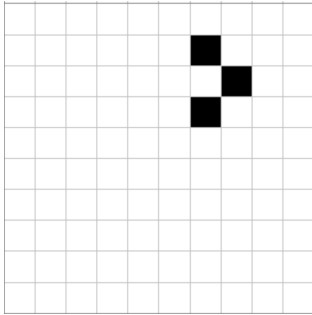
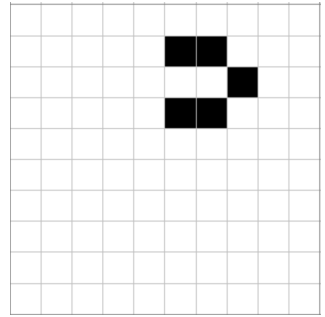
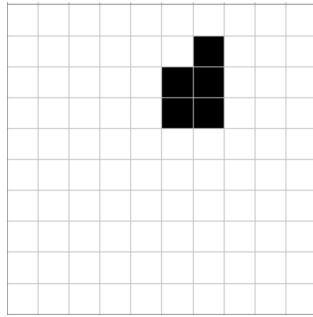
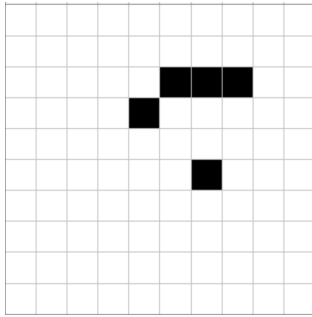
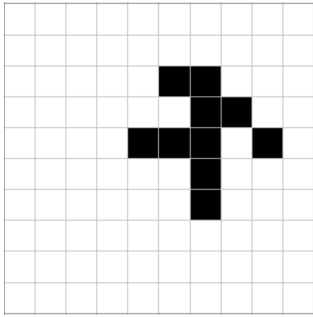
<https://distill.pub/2020/growing-ca/>

The Game of Life — Again

Johnston, Greene.
Conway's Game of Life — Mathematics and Construction.
Online, 2022.

conwaylife.com/book/





Definition 3 (Conway's game of life (standard)). Let $G = (V, E)$ be a graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $surroundings : V \rightarrow \wp(V)$ via

$$surroundings(v) = \{w \mid (v, w) \in E\}$$

so that $|surroundings(v)| = 8$ and $v \notin surroundings(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{dead, alive\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{dead, alive\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in surroundings(v) \wedge x_t(w) = alive\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} dead & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ alive & \text{if } |v|_{x_t} = 3, \\ dead & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.